

Task 8: Read back from on start-up.

- Read your saved records back into the program when the application boots up, use Java's built-in File and Scanner tools.
- Instead of jumping straight into the interactive loop, the program will look for report_cards.txt the exact moment it starts. If the file exists, the program should open it, read it line-by-line, and print the historical records directly to the screen before presenting the registration menu.

What We Are Adding in This Step:

1. **New Package Import:** We add import java.io.File; to manage file path lookups on your operating system.
2. **Boot-Up Initializer Function:** We introduce a new public static method called **loadPastRecords()** outside of main.
3. **The File Reading Process:** We pass a File object into a new Scanner. The scanner uses a while (fileScanner.hasNextLine()) loop to read every single line of text from the file until it hits the very end.
4. **Zero Code Discarded:** The FileWriter append logic, object structure, and error-handling mechanics remain completely active.

How to Test this Evolution:

1. Compile and run the code. If it's your very first time running it with this updated logic, it will say "No historical database found"
2. Add a student (e.g., "Sam Wilson") and let the program log their grades to disk.
3. Exit the application using choice 3.
4. Run the program a **second time**. Notice what happens instantly on boot-up! The loadPastRecords() block finds the file and outputs Sam's entire profile right back onto your console interface before asking you for new inputs.

Future improvements

- Learn about ArrayLists: Upgrade UnifiedTracker.java so that instead of just flashing reports on screen, the system keeps an active in-memory list tracking every student registered during that loop cycle.
- Log error for duplicate entry.
- Add student member variables for student ID, Second name etc.
- Add UI for user interaction.

```

import java.util.Scanner;
import Student_dataHiding.GraduateStudent_inheritance;
import Student_dataHiding.Student_dataHiding;
import java.io.FileWriter;
import java.io.IOException;
// — NEW STEP: IMPORT THE FILE READ UTILITY —
import java.io.File;

// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class Task8_ReadFromFile {

    // — NEW STEP: FUNCTION TO LOAD PAST RECORDS FROM LOCAL DISK —
    public static void loadPastRecords() {
        File databaseFile = new File("report_cards.txt");

        // 1. Check if the backup file actually exists on the computer
        if (!databaseFile.exists()) {
            System.out.println("❗ No historical database found. Starting with a blank registry.\n");
            return; // Exit function early if there is nothing to read
        }

        System.out.println("📁 Loading historical archives from disk drive...");
        System.out.println("=====");

        // 2. Open a specialized scanner locked onto the physical file instead of System.in
        try (Scanner fileScanner = new Scanner(databaseFile)) {

            // 3. Keep loop active as long as there are text strings left to read in the file
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line); // Output historical file lines directly to console
            }
            System.out.println("=====");
            System.out.println("✅ Data successfully loaded into memory view!\n");

        } catch (IOException e) {
            // Guard against system errors during active read runtime
            System.out.println("❌ Local Storage Failure: Could not safely parse text files.");
        }
    }
}

```

```

public static void main(String[] args) {
    // — BOOT-UP REUSE: FETCH OLD HISTORY THE MOMENT APPLICATION STARTS —
    loadPastRecords();

    Scanner inputScanner = new Scanner(System.in);

    System.out.println("=====");
    System.out.println(" WELCOME TO THE GLOBAL STUDENT ENGINE ");
    System.out.println("=====");

    while (true) {
        System.out.println("\nSelect an action:");
        System.out.println("1. Register a Regular Student");
        System.out.println("2. Register a Graduate Student");
        System.out.println("3. Exit Program");
        System.out.print("Enter choice (1-3): ");
        int menuChoice = inputScanner.nextInt();
        inputScanner.nextLine(); // Clear scanner buffer

        if (menuChoice == 3) {
            System.out.println("\nExiting System... Thank you for using Student Engine. Goodbye!");
            break;
        }

        if (menuChoice != 1 && menuChoice != 2) {
            System.out.println(" ❌ Invalid option. Please select 1, 2, or 3.");
            continue;
        }

        System.out.print("\nEnter raw student name: ");
        String rawName = inputScanner.nextLine();

        Student_dataHiding currentStudent = null;

        if (menuChoice == 1) {
            currentStudent = new Student_dataHiding(rawName, inputScanner);
        } else if (menuChoice == 2) {
            System.out.print("Enter Research Thesis Topic: ");
            String thesis = inputScanner.nextLine();
            currentStudent = new GraduateStudent_inheritance(rawName, thesis, inputScanner);
        }

        // --- POLYMORPHIC REPORT COMPILATION ---
    }
}

```

```

System.out.println("\n=====");
System.out.println("      GENERATING REPORT CARD      ");
System.out.println("=====");
System.out.println("Account File:  " + currentStudent.getName());
System.out.println("Tracked Classes: " + currentStudent.getGrades().length);

String extraDetails = "";
if (currentStudent instanceof GraduateStudent_inheritance) {
    GraduateStudent_inheritance castedGrad = (GraduateStudent_inheritance) currentStudent;
    extraDetails = "Research Focus: " + castedGrad.getThesisTitle() + "\n";
    System.out.print(extraDetails);
}

double averageGrade = currentStudent.calculateAverage();
System.out.println("Final Grade:  " + averageGrade + "%");

String honorStatus = "Standard Academic Standing";
if (currentStudent.qualifiesForHonorRoll()) {
    honorStatus = "★ APPROVED FOR HONOR ROLL ★";
}
System.out.println("Honor Roll:  " + honorStatus);
System.out.println("=====");

// AUTOMATIC DATA BACKUP TO HARD DRIVE
try (FileWriter fileWriter = new FileWriter("report_cards.txt", true)) {
    fileWriter.write("=== STUDENT RECORD ===\n");
    fileWriter.write("Name: " + currentStudent.getName() + "\n");
    fileWriter.write("Average: " + averageGrade + "%\n");
    if (!extraDetails.isEmpty()) {
        fileWriter.write(extraDetails);
    }
    fileWriter.write("Status: " + honorStatus + "\n");
    fileWriter.write("-----\n\n");

    System.out.println("💾 Backup Status: Report successfully written to 'report_cards.txt'.");
} catch (IOException e) {
    System.out.println("❌ Local Storage Failure: Could not write record backup to disk drive.");
}

inputScanner.close();
}

```

}